Lab Report: Consistency Models & the CAP Theorem

Course: CSS4004 - Distributed Systems **Topic:** Immutability, PACELC Trade-offs and Quorum Tuning in a Distributed Key-Value Store

System. Three Python/gRPC replicas (A: port 50051 Leaseholder for keyspace [a-z]; B: 50052; C: 50053), extended from the Lab 2 replicated counter (idempotency keys, deadlines/retries and majority-ack writes are reused). Reads are LINEARIZABLE or FOLLOWER_STALE; writes are QUORUM_CP or ALWAYS_AP; divergent copies are merged by Last-Write-Wins ordered by (timestamp, origin node). Network partitions are injected through a SetPartition RPC that makes a node drop all traffic to and from the peers it is cut off from (works for in-process and subprocess nodes).

Section 1: Test Results & Quorum Verification

pytest: 29/29 test cases pass (16 test functions, parametrized cases counted individually) in `tests/test_cap.py`.

Test	Verifies	Passed
quorum intersection formula	R + W > N holds exactly when read and write sets must overlap	8/8
rw quorum experiment matches formula	Exhaustive R/W experiment: zero stale reads iff R + W > N	6/6
cp write on majority partition succeeds	CP Put on DC-1 (A,B): success, acks_received = 2	1/1
cp write on minority partition is rejected	CP Put on DC-2 (C): ReplicaUnavailableError, nothing stored	1/1
cp write succeeds again after heal	After reconnect the same CP Put succeeds with 3 acks	1/1
ap write on minority partition is accepted	AP Put on DC-2: success, acks_received = 1	1/1
partition recovery lww converges to latimestamp	After heal, LWW picks the newest write (both directions)	2/2
lww equal timestamps break ties by node name	Equal timestamps: deterministic tie-break, all replicas agree	1/1
background anti entropy heals without explicit sync	Divergent replicas converge by background anti-entropy alone	1/1
linearizable read from follower is proxied to leaseholder	LINEARIZABLE read sent to a follower is proxied to Leaseholder	1/1
linearizable is strict but follower read stays available	Cut-off follower refuses LINEARIZABLE but serves FOLLOWER_STALE	1/1
leaseholder without majority refuses linearizable reads	Leaseholder that lost its majority refuses linearizable reads	1/1
follower read exceeding max staleness is rejected	STALE_READ_EXCEEDED when contact age > max_staleness_ms	1/1
non leaseholder range key uses majority quorum read	Keys outside [a-z] use a majority quorum read	1/1
idempotency key makes put retries safe	Same idempotency key is applied only once	1/1
subprocess cluster cp vs ap under partition	CP/AP partition behaviour against REAL server.py processes	1/1

1.1 Quorum verification table (N = 3)

The nodes are fully isolated so each replica holds only what the client wrote to it. Every node first holds *old* (timestamp 1). A new version (timestamp 2) is written to **every possible write set of size W**, then read back from **every possible read set of size R** (client keeps the highest timestamp). A read is stale if it misses the new version.

R	W	R + W	R + W > N ?	Write x read sets tried	Stale reads	Outcome
1	1	2	no	9	6 (67%)	stale reads possible
1	2	3	no	9	3 (33%)	stale reads possible

R	W	R + W	R + W > N ?	Write x read sets tried	Stale reads	Outcome
2	1	3	no	9	3 (33%)	stale reads possible
2	2	4	yes	9	0 (0%)	always sees latest write
1	3	4	yes	3	0 (0%)	always sees latest write
3	1	4	yes	3	0 (0%)	always sees latest write

Result: stale reads occur in exactly the configurations with R + W <= N and never when R + W > N, matching the formula.

1.2 Network-partition results (DC-1 = {A, B}, DC-2 = {C})

Step	Request	Expected	Observed
CP-1	Put QUORUM_CP sent to DC-1	success, acks = 2	success=true, acks_received=2
CP-2	Put QUORUM_CP sent to DC-2	failure, quorum loss	ReplicaUnavailableError: quorum lost, reached 1/3, need 2; no replica stored the value
AP-1	Put ALWAYS_AP sent to DC-2	accepted, acks = 1	success=true, acks_received=1
AP-2	Heal partition, then read	LWW merges divergent values	DC-1 wrote v1 (t=100), DC-2 wrote v2 (t=200): all three replicas and the leaseholder return v2. With t=300 on DC-1, all return v1.

Section 2: Performance Evaluation (PACELC)

1000 iterations per scenario (after a 50-read warm-up) against three real server processes on localhost. The client is co-located with follower B. **Linearizable** reads are sent to the Leaseholder A, which is the remote node; **Follower Stale** reads (max_staleness_ms = 5000) are sent to local follower B. Median is statistics.median; p95 is nearest-rank. Latency covers successful reads; availability = successful / total.

2.1 Required table: with 40 ms simulated round trip to the remote leaseholder

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder A (remote)	41.11	41.82	100.0 (1000/1000)
Follower Stale	Local follower B	0.33	0.52	100.0 (1000/1000)

Important: all processes run on one machine, so there is no real WAN. The 40 ms is a *simulated* round trip (time.sleep before each call to the remote node), chosen to represent a cross-datacenter link. The 40 ms figure is therefore my assumption, not a measurement; the local-follower number is a real measurement.

2.2 Same benchmark, no simulated latency (raw localhost)

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder A (remote)	0.35	0.60	100.0 (1000/1000)
Follower Stale	Local follower B	0.36	0.58	100.0 (1000/1000)

Without network distance the two paths cost the same (one gRPC hop plus a dictionary lookup), so the latency gap in 2.1 comes entirely from reaching the Leaseholder, not from extra work in the server.

2.3 Availability when the Leaseholder is partitioned away

Leaseholder A is isolated from B and C for the whole run (1,000 reads per row, 40 ms simulated RTT to A). A lost its majority, so it refuses linearizable reads.

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder A (isolated)	n/a	n/a	0.0 (0/1000)

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Follower Stale (within 5 s bound)	Local follower B	0.36	0.74	100.0 (1000/1000)
Follower Stale (bound expired)	Local follower B	n/a	n/a	0.0 (0/1000)

Reading the results. Follower reads are about 125x faster at the median in the simulated WAN setting and stay available while the leaseholder is unreachable, but only until the staleness bound (5 s) runs out; after that the follower correctly answers STALE_READ_EXCEEDED. Linearizable reads were 100% available when healthy and 0% while the leaseholder lacked a majority. This is the PACELC picture: during a partition (P) the system must pick availability or consistency; otherwise (E) it picks latency or consistency.

Section 3: Conceptual Questions

3.1 Why does R + W > N guarantee up-to-date reads, whereas R + W <= N allows stale reads?

A write is acknowledged by W of the N replicas; a read consults R of them. Two subsets of sizes W and R drawn from N replicas must share at least R + W - N members (pigeonhole principle). If R + W > N that number is at least 1, so every read set contains at least one replica that acknowledged the latest write; taking the highest timestamp among the R answers therefore returns it. If R + W <= N, the sets can be *disjoint*: the read may land entirely on replicas that missed the write and return an old value. Table 1.1 shows exactly this: with N = 3, (R=2, W=2), (1,3) and (3,1) gave 0 stale reads, while (1,1), (1,2) and (2,1) gave stale reads.
Caveat: quorum overlap guarantees that a read sees the latest **completed** write. Full linearizability also needs a single agreed order for concurrent operations and a read that cannot move 'back in time' (e.g. a leader/leaseholder or read repair). This is why the system also offers the leaseholder path, and why sloppy (AP) writes break the guarantee.

3.2 In the partition test, what trade-off was made for writes on the minority partition under QUORUM_CP vs ALWAYS_AP?

QUORUM_CP chose Consistency over Availability. Node C could reach only 1 of 3 replicas, below the majority of 2, so it refused the write (ReplicaUnavailableError) and stored nothing. Clients on the minority side lose write availability, but no replica diverges and no write is ever lost.
ALWAYS_AP chose Availability over Consistency. C accepted the write with a single ack (acks_received = 1), so DC-2 stayed writable, but C and DC-1 held different values for the same key until the partition healed. Convergence then relied on Last-Write-Wins, which resolves the conflict deterministically but **silently discards the older concurrent write** (a lost update) and depends on timestamps. Outside partitions, the PACELC 'else' side shows the same theme: the leaseholder read pays latency for consistency, the follower read pays possible staleness for low latency.

Implementation notes and limitations

(1) A CP write is sent to all reachable peers first and only applied by the coordinator if a majority acknowledged; with N = 3 a failed write leaves no copies, but for larger N a real system would need Raft or 2PC to avoid orphan copies. (2) The Leaseholder is fixed (node A) and there is no automatic lease transfer; when A is cut off, linearizable reads for [a-z] keys are unavailable by design. (3) Staleness is measured as time since the follower last heard from the Leaseholder (heartbeats every 100 ms), a simplification of closed timestamps. (4) LWW needs comparable timestamps; client or server clock skew can make an older write win. (5) The partition injector is a software wrapper inside the nodes, not OS-level packet filtering.